# INDUSTRIAL SURFACE DEFECT DETECTION

*Deep Learning Project Report | BSDA2001P – Introduction to DL and GenAI*

**Bukya Jagadish Naik | Roll No.: 23F3004085**

## Abstract

This project studies pixel-level detection of defects in industrial surface images. The task is formulated as a four-channel, multi-label image-segmentation problem: for each input image, the model predicts a separate binary mask for defect_a, defect_b, defect_c and defect_d. The annotations are supplied as run-length-encoded (RLE) masks and are decoded into pixel masks for training.

The implemented baseline uses a U-Net segmentation architecture with an ImageNet-pretrained ResNet-34 encoder. Images are normalized, masks are decoded, and a PyTorch Dataset/DataLoader supplies image–mask pairs. The training objective combines weighted binary cross-entropy and Dice loss, and experiments are tracked with Weights & Biases (W&B). In the recorded five-epoch W&B run, the best validation Dice was 0.84122, with validation loss 0.26523. A separate threshold study found that the official validation metric was highest at threshold 0.30 (0.5889), while naive mean Dice favoured 0.90 (0.4422). A previously recorded Kaggle score was 0.46864.

The experiments highlight why segmentation evaluation must account for empty masks and use the competition's official metric when selecting a probability threshold. The current documented implementation is a baseline; additional distinct model experiments and an updated leaderboard result must be added before treating this as the final submission report.

## 1. Introduction

Industrial surface inspection requires identifying not only whether a defect exists, but also its location and shape. This project addresses that requirement through semantic segmentation. Given an industrial strip image, the system predicts pixel-level regions for four defect categories. The output is therefore a set of four binary masks rather than one image-level class.

The project objectives are to understand the competition annotations, build a reproducible image-segmentation pipeline, train and evaluate a deep-learning baseline, study class imbalance and decision thresholds, and generate a Kaggle-compatible submission. The course workflow also expects code organization through GitHub, experiment tracking through W&B, and comparison of at least three unique models (including one from scratch and one pretrained model).

This report first describes the dataset and preprocessing, then explains the segmentation pipeline and baseline architecture. It next summarizes the training run, threshold analysis and known Kaggle result, before discussing limitations and the remaining work.

## 2. Dataset and Preprocessing

The competition data is organized under the public dataset directory and contains train.csv, sample_submission.csv, a train image directory and a test image directory. The training annotation table contains 45,572 rows: 11,393 image–class records for each of the four defect categories. There are 11,393 unique training images and 3,000 test images. Each image has a resolution of 800 × 128 pixels. The test set does not require a separate test.csv; test image IDs are taken from the sample-submission file.

The four target channels are defect_a, defect_b, defect_c and defect_d. Each row stores an image_id, defect_type and mask_rle. A missing or empty RLE string represents an empty mask for that image–class pair. The annotation table is reshaped conceptually into one image sample with four corresponding masks so that multiple defect categories can be predicted independently.

RLE decoding creates a zero-initialized array of height × width, fills the foreground runs, and reshapes the result in column-major (Fortran) order. The inverse encoder transposes the binary mask before flattening so that generated predictions follow the competition's RLE convention. Images are loaded with OpenCV, converted from BGR to RGB, scaled to [0, 1], normalized using ImageNet channel means [0.485, 0.456, 0.406] and standard deviations [0.229, 0.224, 0.225], and transposed to channel-first format.

An image-level 80:20 split with random_state=42 produced 9,114 training images and 2,279 validation images. The sample-submission file contains 3,000 unique test image IDs. The baseline notebook uses batch size 8 and returns float32 image tensors and four-channel float32 mask tensors. The documented baseline code does not establish a completed augmentation experiment; augmentation should therefore not be claimed as part of the reported baseline.

| Item | Recorded value |
|---|---|
| Training annotation rows | 45,572 |
| Unique training images | 11,393 |
| Validation images | 2,279 |
| Training images after split | 9,114 |
| Test images | 3,000 |
| Image size | 800 × 128 pixels |
| Defect channels | defect_a, defect_b, defect_c, defect_d |

## 3. Segmentation pipeline and Model

The pipeline follows: Image and CSV loading → image-level train/validation split → RLE decoding → image normalization → PyTorch Dataset/DataLoader → segmentation model → sigmoid probabilities → thresholding → RLE encoding → submission.csv.

The implemented model is U-Net with a ResNet-34 encoder initialized from ImageNet weights. The encoder extracts visual features at multiple spatial scales. The U-Net decoder progressively upsamples these features and combines them with corresponding encoder features through skip connections, helping recover spatial detail needed for thin or small defect

regions. The final head produces four logits per pixel, one for each defect class. A sigmoid is applied independently to each channel because defect classes are represented as separate binary masks.

**Model Architecture :**

```
Input (3 × 128 × 800)
↓
ResNet-34 encoder (ImageNet pretrained)
↓
Multi-scale feature maps
↓
U-Net decoder + skip connections
↓
4-channel pixel logits
↓
Sigmoid → defect_a / defect_b / defect_c / defect_d masks
```

**Training Configuration :**

| Parameter | Baseline setting |
|---|---|
| **Architecture** | U-Net + ResNet-34 encoder |
| **Encoder initialization** | ImageNet pretrained |
| **Input / output channels** | 3 / 4 |
| **Batch size / epochs** | 8 / 5 |
| **Optimizer** | AdamW |
| **Learning rate / weight decay** | 1e-4 / 1e-4 |
| **Loss** | Weighted BCEWithLogitsLoss + Dice loss |
| **Positive class weight** | 10 per defect channel |
| **Random seed** | 42 |
| **Experiment tracking** | W&B: 23f3004085-t32026 |

W&B tracking was configured under project 23f3004085-t32026 with a run named baseline-unet-resnet34. The recorded run summary reports best validation Dice 0.84122 at epoch 5, validation loss 0.26523, training loss 0.26582, learning rate 0.0001 and an epoch time of approximately 128.25 seconds. The run is available at https://wandb.ai/23f3004085-dl-genai-project/23f3004085-t32026/runs/dq5hgfeu.

# 4. Evaluation and results

The project uses overlap-based segmentation metrics. Dice is computed as $2|P \cap G| / (|P| + |G|)$, where P is the predicted foreground and G is the ground-truth foreground. The official competition metric is also evaluated because empty image–class pairs can make an unweighted average misleading. In the reported holdout evaluation, 9,116 image–class pairs contained 1,441 positive pairs and 7,675 empty pairs (84.19%).

**Threshold Comparison :**

| Threshold | Official score | Naive mean Dice | Observation |
|---|---|---|---|
| **0.30** | 0.5889 | Not reported as best | Best official score |
| **0.50** | 0.5870 | — | Close to 0.30 |
| **0.70** | 0.5838 | — | More false positives removed |
| **0.90** | 0.5648 | 0.4422 | Best naive Dice, lower official score |

# 5. Discussion, limitations and future work

The baseline demonstrates a complete segmentation workflow from RLE annotations through training and submission generation. The ResNet-34 encoder provides pretrained visual features, while the decoder restores spatial resolution. Weighted BCE plus Dice loss addresses the low foreground-pixel frequency more directly than an unweighted pixel loss. The threshold study also shows that the choice of metric matters when most image–class pairs are empty.

The main limitations are that the documented evidence currently establishes one trained architecture, and the available results do not provide a complete three-model comparison. The course project criteria require at least three unique models: one built from scratch, one pretrained and one additional model of choice. They also require valid W&B runs and a comparison using common metrics. No verified second or third model result is included in this draft. The reported W&B validation Dice and official competition validation score are different evaluation quantities and should not be compared as if they were the same metric.

Future work should include (1) training a genuinely scratch-built segmentation model, (2) training a second distinct architecture or encoder as the third model, (3) logging all runs with the same validation split and common metrics, (4) adding loss and metric curves from W&B, (5) inspecting false-positive and missed-defect examples, (6) testing connected-component area filtering and test-time augmentation, and (7) retraining/tuning the final model and producing a new Kaggle submission. The final report should be updated with the resulting official score and leaderboard rank.

## References :


1. Ronneberger, O., Fischer, P. and Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI. https://arxiv.org/abs/1505.04597

2. He, K., Zhang, X., Ren, S. and Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR. https://arxiv.org/abs/1512.03385

3. Segmentation Models PyTorch documentation. https://smp.readthedocs.io/

4. PyTorch documentation: BCEWithLogitsLoss and tensor/data loading utilities. https://pytorch.org/docs/stable/

5. Weights & Biases run: baseline-unet-resnet34. https://wandb.ai/23f3004085-dl-genai-project/23f3004085-t32026/runs/dq5hgfeu

6. Kaggle competition dataset: Industrial Surface Defect Detection (public dataset files: train.csv, sample_submission.csv, train/, test/).